

RadioESP32

Introduction

Right at the start, it should be mentioned that this project would probably never have been created without the popular "ESP32Radio-V2" project by Edzelf. I am deeply grateful to the author for his work and thank him for his efforts.

Originally, I adopted the original project without any changes, but very soon it ceased to suit my needs. I was particularly bothered by the fact that the display did not show diacritics (accented characters).

The second major issue for me was the behavior of the SD card file player. One of my requirements for the device was to allow listening to audiobooks. In such a case, however, the files must be played in a defined (i.e., alphabetical) order. But this did not work with the original software. After copying files from a HDD to an SD card, the files always arranged themselves in a somewhat random order, making listening to individual chapters in the correct sequence very complicated (the correct chapter had to be selected manually).

Apart from these two issues, there was a whole range of minor details that I envisioned working differently. For example, the feature that ensured that the last listened-to station always played upon power-up (and with the last used volume level). This did not suit me at all; I prefer the device to play a default station at a default volume when turned on. Furthermore, I planned to add some new features that the original design lacked. Specifically, I wanted the radio to be turned on and off by pressing a single (the same) button, and I wanted it to include a sleep function (i.e., automatic shutdown after a set time).

Despite these facts, I adopted several elements from the original "ESP32Radio-V2" project almost without change, which made my work much easier.

The website design, the method of communication between the ESP32 and the computer (via WebSockets), and several other solutions (such as working with the configuration file) are adopted from the **esp-rfid** project.

Note on Version 2.X.X

I realized that the exact same hardware could function not only as an Internet Radio but also as a convenient Bluetooth Speaker. I finally managed to realize this idea (after many dozens of hours of work), and so from version 2.0.0, the project has been fundamentally redesigned. Because this involved truly massive changes, to simplify things, I dropped the creation of various sub-variants, such as variants without a display or variants with the web server files located in the file system. Now, the release of a new version is limited to a single variant. This variant always includes a display and always stores the web server files in the program memory. Otherwise, it would not even be possible to use the unique solution with the Update Manager (detailed below).

Warning: From version 2.0.0, the minimum required flash memory size is 8MB. The recommended module (minimum configuration) is ESP32-WROVER-E-N8R4.

PSRAM memory is absolutely essential, especially for the Internet Radio application, as newer versions of the ESP32-audioI2S library will not function without it.

Device Architecture

Due to the hardware limitations of the ESP32 chip (problematic simultaneous operation of WiFi and Bluetooth or memory size), I decided on a solution where both applications (Internet Radio and Bluetooth Speaker) are completely separate and reside in their own partitions. As soon as the device receives a command to switch to another application, it automatically performs a reset and boots up as the newly selected application. It is therefore a so-called multi-boot system.

The Bluetooth Speaker application does not use WiFi at all and thus cannot have its own web interface. This is resolved by having both applications share a common configuration file; all configuration (including that pertaining exclusively to the Bluetooth Speaker) is performed within the Internet Radio environment.

As a consequence of having a different application in each "OTA" type partition, it is not possible to use the classic method of installing upgrades (where the new version is saved to the inactive partition and then switched to). Therefore, it was necessary to create a completely new upgrade system. The result is a third, completely independent application called the Update Manager, which is located in the "factory" partition. This application has its own web interface, through which the Internet Radio and Bluetooth Speaker applications can be conveniently upgraded. The Update Manager application itself can only be upgraded "via cable".

The following table clearly shows the layout of the most important flash memory areas:

Flash Memory Area	Application / Data
factory	Update Manager
OTA_0	Internet Radio
OTA_1	Bluetooth Speaker
SPIFFS / LittleFS	Configuration Data

Development with VS Code and PlatformIO

I use VS Code/PlatformIO as my development environment. An important feature of the PlatformIO IDE is that, thanks to the `platformio.ini` file, it is possible to create multiple software variants. This is done using so-called `build_flags`. For the RadioESP32 project, the critical `build_flags` are DATAWEB, SDCARD, BATTERY, and AUTOSHUTDOWN.

- Using the `build_flag` **DATAWEB=true** compiles the software version that has the web server files stored in the "LittleFS" file system area, whereas setting the parameter to false compiles the software version in which the web server files are stored in the program memory (using the PROGMEM attribute) in the form of variables. I call the first version **data** for short, and the second **webh**. The **data** variant is intended exclusively for the development phase, while the **webh** variant is intended for production.
- Using the `build_flag` **SDCARD** enables support for the SD card file player function. Without using this `build_flag`, the Internet Radio application can only play radio stations.
- Using the `build_flag` **BATTERY** enables support for the supply voltage measurement function. This is naturally suitable for battery-powered setups. The display will then show the battery charge level.
- Using the `build_flag` **AUTOSHUTDOWN** enables support for the feature allowing the device to be turned on and off with a single button, as well as the sleep function (adjustable automatic shutdown time). This requires the hardware to be equipped with the necessary circuit.

Three `envs` are defined in the `platformio.ini` file, one for each of the three applications. The default `env` can be chosen by commenting/uncommenting in the `[platformio]` section. During development, however, switching between applications this way is impractical. It is much simpler to use the **Pioarduino: Project Tasks** menu on the left side of the screen. However, this requires the Pioarduino extension to be installed in VS Code. The **Build** action can be found under **Pioarduino: Project Tasks/<application>/General**. The **Upload** action is also located there, but it cannot be used (with the exception of the Internet Radio application). PlatformIO consistently performs uploads to the OTA_0 partition. I bypassed this by creating a script to upload to the correct partition. The script reads the correct partition address directly from the table and then performs the upload (but only for the application, it does not upload anything else). The script can be executed by selecting **Pioarduino: Project Tasks/<application>/Custom/Upload custom binary**. This procedure is the only option available for the Bluetooth Speaker and Update Manager applications. Conversely, the Internet Radio application can also be uploaded in the standard way. Doing this at least once is necessary to ensure that the bootloader and partition table are uploaded as well.

Using the DATAWEB `build_flag`, I proceed with software development as follows:

1. Development takes place exclusively in the `data` software variant (in the `platformio.ini` file, the `build_flag` DATAWEB is set to true). After modifying the source files, compilation is performed in the usual way, and the device is upgraded via the USB (serial) port.
2. If it is necessary to modify the *.html or *.js web server files to fix an error or add a feature, you must open the page http://<device_IP_address>/edit. For this to work, the PC must be connected to the internet, as the online "ace" library (Ajax.org Cloud9 Editor) is used for editing. See <https://ace.c9.io/> for more details.
3. Once the desired functionality is achieved using steps 1 and 2, the `webh` software variant must be created. This is done by running the user script `Download FS & Create WEBH`, which is also available in the `Pioarduino: Project Tasks/<application>/Custom` menu. This transforms the web server files (from the LittleFS file system) into `webh` files. The "pio-esp32-esp8266-file-system-downloader" utility by `maxgerhardt` is used to download and extract the file system image.
4. The entire process ends by switching to the `webh` variant. In the `platformio.ini` file, the `build_flag` DATAWEB must be set to false and compilation performed. The resulting image `<application>_vX.X.X.bin` (created during the compilation process) can be found in the `/bin/<application>` directory. The version designation "X.X.X" is also set before compilation in the `platformio.ini` file as a `build_flag`.

Firmware Installation

The firmware must always be installed onto a new ESP32 (or one previously used in another application) via a cable. There are two options: either directly from the VS Code environment or using the Flash utility.

Installation via VS Code

This method is chosen by experienced users who compile the firmware themselves and can therefore customize it to their needs (either via `build_flags` or by modifying the code).

Due to the multi-boot architecture, the installation procedure is specific. To avoid having to switch the default `env` before compiling and uploading individual applications by editing `platformio.ini` (commenting out lines in the `[platformio]` section), you can advantageously use the `Pioarduino: Project Tasks` sidebar menu. For uploading the `menu_app` and `btls_app` applications, this is the only option!

Procedure:

1. Compile the `radio_app` application using `Project Tasks/radio_app/General/Build`.
2. Upload the `radio_app` application image using `Project Tasks/radio_app/General/Upload`. This will upload not only the application binary but also the partitions.bin and bootloader.bin files.
3. Compile the `btls_app` application using `Project Tasks/btls_app/General/Build`.
4. Upload the `btls_app` application image using `Project Tasks/btls_app/Custom/Upload custom binary`.
5. Compile the `menu_app` application using `Project Tasks/menu_app/General/Build`.
6. Upload the `menu_app` application image using `Project Tasks/menu_app/Custom/Upload custom binary`.

If everything went smoothly, the firmware is installed. You can proceed to the First Boot chapter.

Installation via Flash utility

For less experienced users (who are content with precompiled firmware), I have prepared

the Flash utility script.

Warning: Before running the installation, make sure that the selected COM port is not being used by another program (e.g., the serial monitor in VS Code), otherwise `esptool.exe` will fail to connect to the chip.

Installation procedure (assuming the ESP32 is connected to the computer via a USB cable):

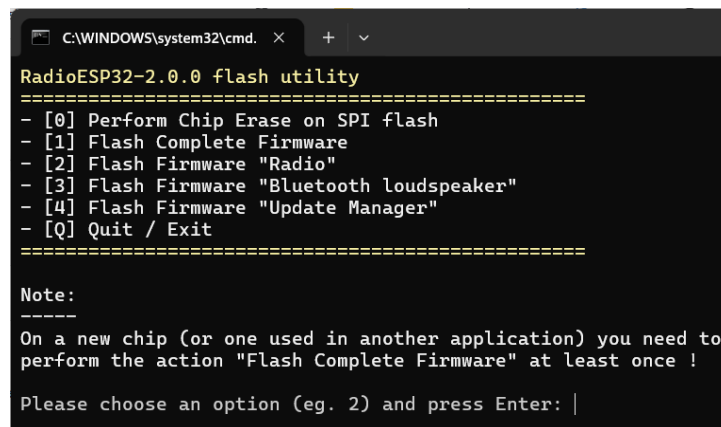
1. Download the installation package:
https://github.com/Pako2/RadioESP32/releases/latest/download/RadioESP32_binaries.zip
2. Extract the downloaded package into a suitable folder - let's call it RadioESP32, for example.
3. Open Windows Explorer and navigate to the chosen folder (RadioESP32).
4. Run (e.g., by double-clicking) the `flash.bat` file. This automatically launches a more modern PowerShell script, `flash_utility.ps1` (Windows security will safely bypass the script restriction), and the menu shown below will appear.
5. Recommended (optional) step – option **[0] – Perform Chip Erase on SPI flash**.
6. Mandatory step – option **[1] – Flash Complete Firmware** (if step 5 was also performed, you must run `flash.bat` again).

If everything went smoothly, the firmware is installed. After successful flashing, I recommend disconnecting and reconnecting the USB cable (or pressing the EN/RST button on the module). Sometimes, a physical reset is required for the ESP32 after an upload to correctly initialize the partition table and LittleFS.

Notes:

Options **[2]** to **[4]** are not intended for the initial installation phase. They can subsequently be used to perform upgrades of individual applications (if for some reason the procedure via the Update Manager is not preferred).

Option **[Q]** serves to safely exit the utility if the user changes their mind about flashing.



```
C:\WINDOWS\system32\cmd. x + v
RadioESP32-2.0.0 flash utility
=====
- [0] Perform Chip Erase on SPI flash
- [1] Flash Complete Firmware
- [2] Flash Firmware "Radio"
- [3] Flash Firmware "Bluetooth loudspeaker"
- [4] Flash Firmware "Update Manager"
- [Q] Quit / Exit
=====

Note:
-----
On a new chip (or one used in another application) you need to
perform the action "Flash Complete Firmware" at least once !

Please choose an option (eg. 2) and press Enter: |
```

First Boot

When the device is booted for the first time, it runs with default settings and in AP mode. On a PC, connect to the "RadioESP32" network and navigate to the address 192.168.4.1 in a browser. A web page should appear, through which the device configuration must be carried out. The description of the web interface is provided in the Main Menu chapter.

Upgrade

The Internet Radio and Bluetooth Speaker applications can be upgraded very easily via the web interface of the Update Manager application.

Procedure:

- a) You can switch to the Update Manager application from the Internet Radio and Bluetooth Speaker applications using the display menu, which can be invoked by long-pressing the rotary encoder knob. The second option is to press the software button [Jump to Update Manager](#), located on the "Check for updates" page. This, of course, applies only to the Internet Radio.
- b) After successfully switching applications, the red Update Manager interface will appear instead of the blue Internet Radio web server interface. To enter it, you must enter the password "admin" (as defined in the source text of the Update Manager application).
- c) In the menu (on the left side of the screen), select the item **Update Manager**.
- d) A table will appear, displaying both the current versions of the applications and the versions available on the internet.
- e) If a newer version exists compared to the current one, you can press the software **Update** button and then follow the instructions on the screen. The result should be an upgraded application. Additionally, on the same page, there are buttons that can be used to create backups of the current applications (.bin files). Furthermore, there is also the option to upload (and thus perform a potential downgrade to) any other .bin file available on the computer's disk.

Upgrade from Version 1.X.X to Version 2.X.X

Since the architecture of both versions is completely different, a full installation is required.

Recommended procedure:

- Backup your configuration (via the Backup & Restore menu).
- Perform the upgrade - for example, by following the Installation via Flash utility chapter.
- Restore your configuration (via the Backup & Restore menu).

Main Menu

The main menu is always on the left side of the screen. Description of the main menu items:

Radio Player

Clicking this item opens a page from which the radio can be comfortably controlled.

SD Card Player

This item is only displayed if the software was compiled with the **SDCARD** option enabled. Clicking this item opens a page from which the functions of the SD card file player can be comfortably controlled.

Status

Clicking this item opens a page showing basic data about the device.

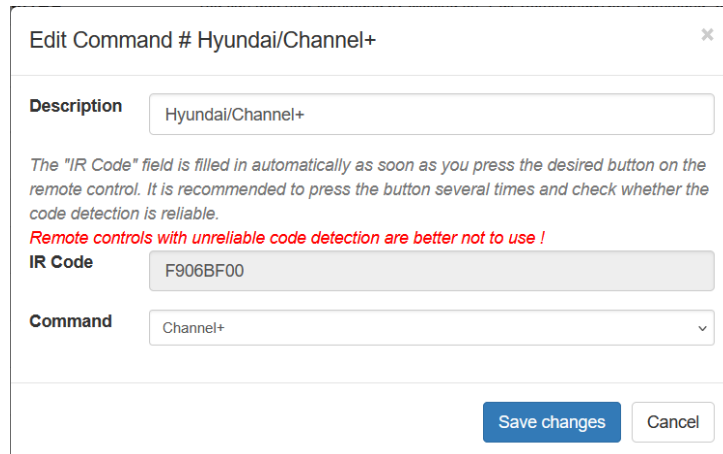
Settings

If you need to adjust the device settings (for example, during the first boot), select the **Settings** item. Clicking it expands the menu to show nine sub-menus:

- **Radio Settings:** This page is primarily used to create the list of radio stations. Furthermore, you can set the default station here (which is automatically selected when the device is turned on), the default volume, and simple equalizer values.
- **SD Card Settings:** This item is only displayed if the software was compiled with the **SDCARD** option enabled. Only one parameter can be set on this page, which is "Seek step". This is the jump length (forward and backward) in seconds. The default volume and equalizer settings configured on the **Radio Settings** page also apply to the SD card file player.
- **Bluetooth Settings:** Some properties of the Bluetooth Speaker can be configured on this page (for example, the name by which the Bluetooth Speaker presents itself).
- **Network Settings:** The network (or multiple networks) to which the device should connect must be

configured on this page. Press [Edit Networks](#) and then [New Network](#). In the dialog box that appears, set the required values. The [Scan](#) button can be used to search for available networks. After configuration, press [Save Changes](#). Below the network table, it is also possible to change the SSID, IP address, and mask (all in AP mode). The changes must be confirmed by pressing the [Save](#) button and subsequently the yellow button containing the text [You have uncommitted changes, please click here to commit and reboot ...](#). However, it is advisable to use this yellow button only after reviewing all other settings, as the device will restart after its use.

- **Hardware Settings:** On this page, it is possible to configure some of the GPIO pins used (some are set before compilation in `platformio.ini`) and several other parameters. The configured values must again be confirmed by pressing the [Save](#) button.
- **IR Remote Settings:** Remote control commands are configured on this page. Adding a new command is very simple and intuitive. Configured commands must again be confirmed by pressing the [Save](#) button.



The screenshot shows a dialog box titled "Edit Command # Hyundai/Channel+" with a close button (X) in the top right corner. Inside the dialog, there is a "Description" field containing "Hyundai/Channel+". Below this, there is a note: "The 'IR Code' field is filled in automatically as soon as you press the desired button on the remote control. It is recommended to press the button several times and check whether the code detection is reliable." followed by a red warning: "Remote controls with unreliable code detection are better not to use !". Below the note, there is an "IR Code" field containing "F906BF00". At the bottom, there is a "Command" dropdown menu showing "Channel+". At the very bottom of the dialog, there are two buttons: "Save changes" and "Cancel".

- **Display Settings:** This page allows you to set, in particular, the display backlight levels and the idle clock display settings. The user can also name the days of the week in their own language (by default, the names are in English). As always, the [Save](#) button must be pressed after all changes.
- **General Settings:** It is possible to set the "Machine Name" (Host Name) on this page, and the battery charge level display is also calibrated here. It is confirmed again by pressing the [Save](#) button.
- **NTP (Time) Settings:** The NTP server, synchronization interval (in minutes), and time zone can be configured on this page. Changes are confirmed again by pressing the [Save](#) button. Sometimes it may happen that when the device is turned on (or restarted), none of the known WiFi networks are available. In such a case, it is obviously not possible to synchronize the time with the NTP server. In this extreme situation, the [Sync Browser Time to Device](#) button will help.

Backup & Restore

In the main menu, we further find the item "Backup & Restore". The meaning is obvious from the name. It is possible to save a file with a backup of the settings to the PC and easily return to the same settings if necessary. Another function is located under the specified item – rebooting the device.

Factory reset

The function of the same name is hidden under this item of the main menu. After its activation, the configuration file is deleted, and after a reboot, the device is in the same state as during the first boot.

Check for updates

If we select this item, a table with the current and available versions of individual applications will appear. It is possible to press the [Jump to Update Manager](#) button and switch to the respective application. The subsequent procedure was described in the [Upgrade](#) chapter.

Controlling the Device via the Rotary Encoder

Internet Radio Application

- In the **basic mode**, turning the knob controls the volume, and pressing it toggles **Mute** on/off.
- **Double-pressing** the knob switches the device to the radio station selection mode. The preset list is scrolled by turning the knob. The selected station must be confirmed by pressing the knob.
- **Triple-pressing** the knob switches the device to the SD card file selection mode. Turning the knob scrolls through the track list. The selected file must be confirmed by pressing the knob.
- **Long-pressing** the knob (about 4 seconds) invokes the application switching menu. Additionally, this menu includes an item that allows turning the device off.

Bluetooth Speaker Application

- In the **basic mode**, turning the knob controls the volume. The function assigned to pressing the knob can be selected in the Bluetooth Speaker properties settings (via the radio's web interface). By default, pressing the button toggles between **Pause** and **Play**. The second option (which must be selected in the configuration) is toggling **Mute**.
- **Double-pressing** the button switches the device to the track "skipping" mode (forward or backward). In the default state, pressing the knob controls **Mute**, while the second option (which must be selected in the configuration) is controlling **Pause/Play**.
- **Long-pressing** the knob (about 4 seconds) invokes the application switching menu. Additionally, this menu includes an item that allows turning the device off.

Update Manager Application

In the Update Manager application, the rotary encoder only controls the menu for application selection (or turning off). It has no other function.

Controlling the Device via Infrared Remote Control

In the Main menu/Settings/IR Remote Settings chapter, it is described how the device can be taught remote control commands. Commands are shared between the Internet Radio application (including the SD card player) and the Bluetooth Speaker application. The Update Manager application cannot be controlled via IR. Control does not differ from regular multimedia devices and does not need to be described in further detail.

Note:

When entering a preset by number, it is not necessary to confirm the number with the OK command – just wait a brief moment. In contrast, when entering a sleep time, the entered time must be confirmed with the OK command.

Device Hardware Design

The construction is sufficiently described in the README.md file within the project repository. It can be viewed at: <https://github.com/Pako2/RadioESP32/blob/main/README.md>.